The DSL, or: one file that replaced a product's engine

A Ruby agent harness with a fabric, an identity, a memory and a face

Runes 0.4.0 | 11 November 2026 | github.com/La-Voix-du-chat-artiste/runes

What it feels like to write a workflow that does the work of a small SaaS - and why the fun is not an accident.

The 60-second version

`examples/prospect_pipeline.rb` is **431 lines of Ruby-ish DSL**. It reads an idea, writes a goal, plans a mission as a Mermaid kanban, executes every todo and has a *second* model call verify each one, moves the cards, computes who to contact next by a rule, drafts the outreach without sending it, and writes a weekly report. Every artifact is a file another tool can read.

```
bin/runes-workflow execute examples/prospect_pipeline.rb idea="Trouver 10 PME"
```

It runs offline, in the test suite, in about **50 milliseconds** - with the provider seam scripted - and that test asserts what it wrote: the kanban validates against another project's own validator, one todo lands in Done, one in Blocked *with its reason preserved*, the report carries the failure, and the references it wrote resolve.

There is no framework to learn, no migration to run, no queue to supervise, no service object to name. There is a file, seven verbs, and the artefacts.

The whole DSL, on one page

Seven verbs. If you know Shopify Roast, you already know them; that is the point of matching its DSL byte for byte.

Verb	What it is for
`cmd`	run a command; `cmd!(:x).out` / `.lines` / `.status`
`ruby`	pass any Ruby value through — the escape hatch, and how a workflow touches a file
`agent`	a tool-using agent turn; `agent!(:x).response`
`chat`	one completion, no tools; `chat!(:x).response`
`map`	run a named sub-workflow **once per item**
`repeat`	run a named sub-workflow N times (or until it stops changing)
`call`	run a named sub-workflow once

The rules that make it feel small:

```
ruby(:config) do |_my, scope_value, _index|   # self is the step's input context
kwargs = scope_value.respond_to?(:kwargs) ? scope_value.kwargs : {}
{ idea: kwargs[:idea] || ENV["PROSPECT_IDEA"] }   # the block's value is the output
end

agent(:goal) { "Turn this into a decided goal: #{ruby!(:config).idea}" }

outputs { |_value, _index| { goal: agent!(:goal).response } }
```

- `X(:name)`, `X!(:name)`, `X?(:name)` - the output, the output or a raise, the output or `nil`. `ruby!(:config).idea` also works on a Hash value.
- `from(call!(:scope))` - a sub-workflow's declared outputs.
- `collect(map!(:x))` - the outputs of every iteration; `reduce(map!(:x), 0) { }` folds them.
- `fail!("...")` stops the run with your message; `skip!`, `next!`, `break!` do what you hope.
- `params` (positional targets, symbol flags, `key=value`) come from the CLI.

That is the entire surface. Everything else in the example is domain logic.

The pipeline, scope by scope

Six scopes, each one thing. This is the whole implementation review:

Scope	What it does	The DSL it uses
`:prepare`	reads params, mints `#E-001` from the epics on disk, stores the raw idea as a content-addressed document (`ab/cd/sha256>.txt`), creates the epic folder	`ruby`
`:brainstorm`	one agent call → `goal.md`, with a machine-readable `!-- code: E-001 -->` marker and a link check that reports references that point nowhere	`agent`, `ruby`, `outputs`
`:plan`	one agent call returning JSON → `Runes::Kanban.render` writes the mission `.mmd`, then **validates it** and `fail!`s if the grammar is wrong	`agent`, `ruby`
`:advance_todo`	per todo: work → verify → move the card under a file lock, keeping the verdict reason as free text	`ruby`, `agent`, `outputs`
`:crm_actions`	a deterministic rule for "who to contact next", then one drafted message per contact through a `map`	`ruby`, `map`
`:weekly_report`	a `chat` summary, plus `Runes::Index` resolving the `#E-001`/`#M-001` the report cites	`chat`, `ruby`

The interesting part is not the length. It is that **the 431 lines contain no infrastructure**: no retry loop, no queue, no idempotency key, no audit trail, no step state machine, no "what if it crashes here". Those are properties of the engine, and they are the same properties whether your workflow has six steps or sixty.

Three sharp edges (so you don't find them the way I did)

1. **`from(call!(:x))` returns the raw value, not a rune output.** `plan.todos` fails; `plan[:todos]` works. (`ruby!(:x).key` *does* work - the Ruby rune's output delegates to its value - and that asymmetry is the trap.) 2. **`collect(map!(:x))` returns whatever the sub-scope's `outputs` block returned** - so don't wrap it in `.map(&:value)` unless your sub-scope returned output objects. It also cannot be called from inside a `call` block (there are no iteration managers in that context): read a map's results in a `ruby` step. 3. **Key types are a real hazard.** A model returns JSON (string keys); Ruby builds symbols. Symbolise once at the boundary (`JSON.parse(text, symbolize_names: true)`) and the report stops being silently empty.

All three are now in the test suite as behaviour worth pinning, and in the example as comments where you will hit them.

Why it is fun

- **The loop is 50 ms.** `test/prospect_pipeline_test.rb` runs the real file offline against a scripted provider. Change a prompt, run one test, see the artefact - no server, no seed data, no waiting for a queue.
- **The artefacts are files you can open.** `goal.md`, a `.mmd` kanban, a `people.json`, drafts, a report. You review a workflow run the way you review a pull request.
- **The DSL is Ruby.** `puts`, `pp`, `binding.irb`, a method you extract, a helper module - it is all available, because a workflow is `instance_eval`'d Ruby rather than a YAML dialect.
- **A verb is 30 lines.** `Runes::Plugins::Greet < Runes::Rune` with an `Input`/`Output` pair and an `execute`, and `greet(:hi) { "world" }` exists in every workflow.
- **Interop is the default.** The same file runs under `bin/runes-workflow`, and the verbs are Roast's, so a Roast file runs here unmodified - the shipped example *is* Roast's README example, byte for byte, and the suite reads that file.
- **The formats are someone else's.** This pipeline writes the `.mmd` kanban grammar that `pipeline_prospect` publishes, so the file it produces can be read by a human, edited by a harness, and synced by that app. Nothing here invented a format to be interesting.
- **You can watch it.** `RUNES_TELEMETRY=mqtt` sends the run's step telemetry to the observatory, where `/board` shows the steps as Working/Done and `GET /board.mmd` hands the same diagram to any program.

Why it is safe to leave running

A workflow is a program that touches the world. The engine gives it the things a cron job does not have:

Property	Where it comes from
A redelivered or retried request runs once	`Runes::RequestLedger` (bounded, TTL'd, atomic)
A crash mid-run is resumable, and the history is on disk	the journal
A refusal is an event someone can count, not a silent `false`	`Runes::GuardTelemetry`, `/security`
The run is watchable, and its diagram is machine-readable	`/board`, `board.mmd`
The whole conversation can be replayed offline	`bin/runes-replay`
A file two writers share is not clobbered	`Runes::Kanban.update_file` takes `flock` for the read-modify-write
Model text cannot restructure a file	titles and headers are sanitised before they are written
Identical content is one file, addressed by its hash	`Runes::DocStore`

The last four are new in this work, and they exist because writing the pipeline exposed them: a hostile title could forge a checkbox, two writers could lose a move, and a reference at the end of a Markdown emphasis was silently dropped by a `\b` in a regex. That is the honest advertisement for a DSL: the engine is small enough to review, and the review found real bugs.

Tweak it in a line

You want	Change
a different size of plan	the planner prompt: *"between three and six todos"* → *"exactly two"*
different people contacted	`first(2)` → `first(5)`
your own next-action rule	one `elsif` in the rule chain
no verifier for a trusted step	swap the `agent(:verdict)` step for a `ruby` returning a pass
send instead of draft	replace one `File.write` with your channel
a weekly run	name the top scope, `call` it from a `repeat`
your folder shape	two `File.join` roots in `:prepare`
a different provider	`RUNES_LLM_ADAPTER`, or the provider env — no code

What it is not

It is not a product. No auth, no tenancy, no billing, no GDPR register, no landing page, no admin console - those are obligations, not intelligence, and they belong in an app. It is not a two-way sync (the workflow writes mission files; reading back a human's edit mid-run is the app's job). And it is not magic: the verifier is a second model call, not a proof, and the outreach stays a draft because sending has a human's name on it.

That is the offer: keep the surface you need, and make the engine a file you can read, test in a second, watch on a board, and change before lunch.

Companion documents: `WHY_RUNES_V2.md` for why any of this matters, and `EXAMPLE_CRM_PIPELINE.md` for the pipeline's receipts and boundaries.